Referring to the previous chapter, this breaches the dependency inversion principle because the context is not an abstraction. Your `PageModel` class is tightly coupled to the data access technology you have chosen, and if you instantiate an instance of this class in a unit test, for example, it will call into whatever database is defined in the context's connection string. That's not a unit test. It's an integration test. Ideally, you should replace the context with a service or, rather, its abstraction.

The next major issue with the generated code opens up a potential attack vector. The generated code makes the entire `Property` class a binding target:

```
[BindProperty]
public Property Property { get; set; }
```

If you recall from chapter 5, you should limit the scope of the binding targets on your page to just those properties you want your users to set. If you expose more, your application is susceptible to overposting attacks. You do actually want to expose all the *current* properties of the `Property` class to the model binder, but that may not always be the case. In the future, you might add more properties to the class. They will be exposed to model binding by default because you decorated the entire class with the `BindProperty` attribute. So as a best practice, you should start by explicitly exposing properties individually or through an `InputModel`. The next listing shows the scaffolded `OnGetAsync` handler method.

Listing 8.23 The scaffolded `EditModel` `OnGetAsync` handler

```
public async Task<IActionResult> OnGetAsync(int? id)
{
    if (id == null)
    {
        return NotFound();
    }
```